# LAPORAN PRAKTIKUM
# PEMROGRAMAN BERORIENTASI OBJEK
# MODUL 07 – Abstract & Interface

MUHAMMAD ‘IZZUDDIN NABIL

103132400032

PS1DS-05-REG-01

**PROGRAM STUDI SAINS DATA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

A. Dasar Teori

Kelas abstrak adalah kelas yang dideklarasikan dengan kata kunci abstract dan dapat memuat method abstrak (method tanpa body) maupun method konkrit. Karakteristik penting kelas abstrak antara lain:

- Tidak Dapat Diinstansiasi: Objek tidak dapat dibentuk secara langsung dari kelas abstrak (misalnya perintah new TiketBioskop(...) akan menghasilkan compile error). Objek hanya dapat dibentuk melalui kelas konkrit yang mewarisinya.
- Pewarisan Tunggal (Single Inheritance): Sebuah kelas turunan hanya diperbolehkan mewarisi satu kelas induk menggunakan kata kunci extends.
- Implementasi Parsial (Partial Implementation): Kelas abstrak dapat memiliki atribut data (state), konstruktor, serta method konkrit dengan implementasi penuh, sembari memaksakan kontrak method abstrak yang wajib di-override oleh kelas anaknya.

Interface adalah kontrak perilaku murni (pure behavioral contract). Relasi antara kelas dengan interface dinamakan realisasi (realization) yang diimplementasikan menggunakan kata kunci implements:

- Mendukung Multiple Realization: Berbeda dengan class biasa, sebuah kelas dalam Java dapat mengimplementasikan banyak interface sekaligus, sehingga mengatasi batasan single inheritance pada hierarki kelas.
- Penyatuan Entitas Lintas Hierarki: Interface memungkinkan penggabungan kelas-kelas yang tidak saling berkaitan secara silsilah keluarga (misalnya tiket nonton dan paket snack) agar dapat diproses secara seragam.
- Evolusi Fitur Java 8+ (Default & Static Method): Pada Java 8+, interface diperkaya dengan default method (metode konkrit yang dapat langsung diwariskan tanpa memecah kelas lama) dan static method (metode utilitas independen yang dipanggil langsung melalui nama interface).

B. Guided

1. Kelas Abstract

```java
package guided.gd1;

public abstract class Employee {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
```

```
    // every employee must implement earnings method but different
employees will have different implementations
    public abstract double earnings();
}
```

```
package guided.gd1;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }

}
```

```
package guided.gd1;

public class Main {

    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Super Max",
1000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-07-abstract-dan-interface>  & 'C:\Users\Lenovo\
32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' '
a0ec46c83f1980e7667a248df55\redhat.java\jdt_ws\modul-07-abstract-dan-inte
Nama: Super Bil
Gaji: Rp1000000.0
```

2. Interface

```java
package guided.gd2;

public interface Payable {
    double getPaymentAmount();
}
```

```java
package guided.gd2;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // every employee must implement earnings method but different
employees will have different implementations
    public abstract double earnings();

    // if there's a due payment, the amount will be the same as the
earnings of the employee
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```java
package guided.gd2;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
```

```
    public double earnings() {
        return weeklySalary;
    }

}
```

```
package guided.gd2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

```
package guided.gd2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];

        items[0] = new Invoice("Monitor LED", 2, 150000);
        items[1] = new SalariedEmployee("Super Max", 5000000);

        for (Payable item : items) {
            System.out.println("Jumlah yang perlu dibayar: " +
item.getPaymentAmount());
```

```
        }
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-07-abstract-dan-interface>  d:; cd 'd:\Praktiku
de\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\
ers\Lenovo\AppData\Roaming\Code\User\workspaceStorage\74e52a0ec46c83f1980
_74edf15b\bin' 'guided.gd2.Main'
Jumlah yang perlu dibayar: 300000.0
Jumlah yang perlu dibayar: 5000000.0
```

3. Interface

```
package guided.gd3;
public interface Payable {
    double getPaymentAmount();

    // fungsi baru
    default void printReceipt() {
        System.out.println("Struk Resi - Total: Rp" +
getPaymentAmount());
    }

    // fungsi utilitas
    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

```
package guided.gd3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // every employee must implement earnings method but different
employees will have different implementations
```

```
    public abstract double earnings();

    // if there's a due payment, the amount will be the same as the
earnings of the employee
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```
package guided.gd3;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }

}
```

```
package guided.gd3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
```

```
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

```
package guided.gd3;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("RAM DDR5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Super Max", 5000000);

        for (Payable item: items) {
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");

    }
}
```

Output:

```
PS D:\Praktikum PBO\modul-07-abstract-dan-interface>  d:; cd 'd:\Praktiku
de\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\
ers\Lenovo\AppData\Roaming\Code\User\workspaceStorage\74e52a0ec46c83f1980
_74edf15b\bin' 'guided.gd3.Main'
[AUDIT LOG] Sistem pembayaran diinisialisasi.
Struk Resi - Total: Rp4000000.0
Struk Resi - Total: Rp5000000.0
[AUDIT LOG] Transaksi selesai diproses.
```

C. Unguided

1. Soal Unguided 1

Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli Tiket Nonton (Tiket Reguler atau Tiket VIP), tetapi juga dapat membeli makanan/minuman (Paket Snack F&B). Tiket Bioskop dan Paket Snack berasal dari hierarki keluarga yang sepenuhnya

berbeda, namun harus dapat disatukan, dihitung total tagihannya, dan dicetak dalam satu struk transaksi secara otomatis melalui kontrak antarmuka.
Ketentuan Teknis Program:

- **Antarmuka ItemTransaksi:** Mendeklarasikan kontrak metode abstrak double getNominalBayar() dan String getNamaTransaksi(). Dilengkapi dengan default method cetakRincianItem() untuk mencetak baris struk "- [Nama Transaksi] | Total: Rp[Nominal Bayar]" serta static method catatLogKasir(String pesan) untuk pencatatan log sistem "[LOG SISTEM KASIR] [pesan]".
- **Kelas Abstrak TiketBioskop:** Mengimplementasikan ItemTransaksi. Menyimpan atribut privat judulFilm, hargaDasar, dan jumlahKursi beserta konstruktor (this) dan getter/setter. Memiliki abstract method hitungTotalBayar() dan getNamaTransaksi(), serta merealisasikan getNominalBayar() yang mengembalikan hasil hitungTotalBayar().
- **Kelas Konkrit TiketReguler:** Mewarisi TiketBioskop dengan atribut tambahan nomorStudio. Menghitung total bayar (hargaDasar * jumlahKursi) dan menghasilkan format nama transaksi "Tiket Reguler (Studio [nomor]) - [Film] ([kursi] x Rp[harga])".
- **Kelas Konkrit TiketVIP:** Mewarisi TiketBioskop dengan atribut tambahan biayaFasilitasVIP per kursi. Menghitung total bayar ((hargaDasar + biayaFasilitasVIP) * jumlahKursi) dan menghasilkan format nama transaksi "Tiket VIP Premiere - [Film] ([kursi] x Rp[totalHargaPerKursi])".
- **Kelas Konkrit PaketSnack:** Mengimplementasikan ItemTransaksi (bukan turunan tiket) dengan atribut privat namaPaket, jumlahPorsi, dan hargaPerPorsi. Menghitung nominal bayar (jumlahPorsi * hargaPerPorsi) dan menghasilkan format nama transaksi "F&B Snack: [Paket] ([porsi] x Rp[harga])".
- **Kelas Driver Main:** Menguji proteksi kelas abstrak (instansiasi TiketBioskop dijadikan komentar), mencatat log buka kasir Pelanggan #042, menampung ketiga objek dalam array polimorfik ItemTransaksi[], mengeksekusi cetakRincianItem() dalam perulangan, menghitung grand total, dan menutup transaksi via log kasir.

Jawaban :
Interface

```
package unguided;

public interface ItemTransaksi {
    // Behavior
    double getNominalBayar();
    String getNamaTransaksi();

    default void cetakRincianItem() {
```

```
        System.out.println("- " + this.getNamaTransaksi() + " |
Total: Rp" + this.getNominalBayar());
    }

    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

Abstract Class

```
package unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    // Constructor
    public TiketBioskop(String judulFilm, double hargaDasar, int
jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    // Behavior
    public abstract double hitungTotalBayar();
    public abstract String getNamaTransaksi();

    public double getNominalBayar() {
        return this.hitungTotalBayar();
    }

    // Getter
    public String getJudulFilm() {
        return this.judulFilm;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public int getJumlahKursi() {
        return this.jumlahKursi;
```

```
    }

    // Setter
    public void setJudulFilm(String judulFilm) {
        this.judulFilm = judulFilm;
    }

    public void setHargaDasar(double hargaDasar) {
        this.hargaDasar = hargaDasar;
    }

    public void setJumlahKursi(int jumlahKursi) {
        this.jumlahKursi = jumlahKursi;
    }
}
```

Concrete Subclass

```
package unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    // Constructor
    public TiketReguler(String judulFilm, double hargaDasar, int
jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    // Behavior
    public double hitungTotalBayar() {
        return this.getHargaDasar() * this.getJumlahKursi();
    }

    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + this.nomorStudio + ") - " +
this.getJudulFilm() + " (" + this.getJumlahKursi() + " x Rp" +
this.getHargaDasar() + ")";
    }

    // Getter
    public int getNomorStudio() {
        return this.nomorStudio;
    }
```

```
    // Setter
    public void setNomorStudio(int nomorStudio) {
        this.nomorStudio = nomorStudio;
    }
}
```

Concrete Subclass

```
package unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    // Constructor
    public TiketVIP(String judulFilm, double hargaDasar, int
jumlahKursi, double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    // Behavior
    public double hitungTotalBayar() {
        return (this.getHargaDasar() + this.biayaFasilitasVIP) *
this.getJumlahKursi();
    }

    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + this.getJudulFilm() + " (" +
this.getJumlahKursi() + " x Rp" + (this.getHargaDasar() +
this.biayaFasilitasVIP) + ")";
    }

    // Getter
    public double getBiayaFasilitasVIP() {
        return this.biayaFasilitasVIP;
    }

    // Setter
    public void setBiayaFasilitasVIP(double biayaFasilitasVIP) {
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }
}
```

Concrete Subclass

```java
package unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    // Constructor
    public PaketSnack(String namaPaket, int jumlahPorsi, double
hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    // Behavior
    public double getNominalBayar() {
        return this.jumlahPorsi * this.hargaPerPorsi;
    }

    public String getNamaTransaksi() {
        return "F&B Snack: " + this.namaPaket + " (" +
this.jumlahPorsi + " x Rp" + this.hargaPerPorsi + ")";
    }

    // Getter
    public String getNamaPaket() {
        return this.namaPaket;
    }

    public int getJumlahPorsi() {
        return this.jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return this.hargaPerPorsi;
    }

    // Setter
    public void setNamaPaket(String namaPaket) {
        this.namaPaket = namaPaket;
    }

    public void setJumlahPorsi(int jumlahPorsi) {
        this.jumlahPorsi = jumlahPorsi;
```

```
    }

    public void setHargaPerPorsi(double hargaPerPorsi) {
        this.hargaPerPorsi = hargaPerPorsi;
    }
}
```

Main

```
package unguided;

public class Main {
    // Behavior
    public static void main(String[] args) {
        // TiketBioskop tiket = new TiketBioskop("Interstellar",
40000, 2);

        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk
Pelanggan #042.");

        ItemTransaksi[] keranjang = new ItemTransaksi[3];
        keranjang[0] = new TiketReguler("Interstellar", 40000, 2, 3);
        keranjang[1] = new TiketVIP("Dune: Part Two", 60000, 2,
25000);
        keranjang[2] = new PaketSnack("Combo Popcorn Caramel + 2
Soda", 1, 55000);

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");

        double totalTransaksi = 0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            totalTransaksi += item.getNominalBayar();
        }

        System.out.println("----------------------------------------
--------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + totalTransaksi);
        System.out.println("========================================
====================");

        ItemTransaksi.catatLogKasir("Transaksi #042 selesai
diproses.");
    }
}
```

Output :

```
PS D:\Praktikum PBO\modul-07-abstract-dan-interface>  d:; cd 'd:\Praktikum PBO\
de\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.e
ers\Lenovo\AppData\Roaming\Code\User\workspaceStorage\74e52a0ec46c83f1980e7667a
_74edf15b\bin' 'unguided.Main'
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Penjelasan :

- **ItemTransaksi.java (Interface):** Berperan sebagai kontrak antarmuka publik yang menjembatani entitas tiket dan makanan/minuman agar dapat dikelola secara seragam oleh sistem kasir. Mendeklarasikan method abstrak getNominalBayar() dan getNamaTransaksi(). Menggunakan default method cetakRincianItem() untuk mencetak format baris rincian struk secara terpusat dan dinamis, serta static method catatLogKasir() untuk pencatatan log transaksi kasir.
- **TiketBioskop.java (Abstract Class):** Berperan sebagai fondasi umum tiket film yang berstatus abstract sehingga terlindungi dari pembentukan objek secara langsung (instansiasi dicegah). Mengimplementasikan ItemTransaksi dengan menjembatani getNominalBayar() ke method abstrak hitungTotalBayar(). Menerapkan enkapsulasi penuh (private fields: judulFilm, hargaDasar, jumlahKursi) dengan explicit this pada konstruktor, serta mewajibkan turunan studio untuk merumuskan perhitungan biaya dan deskripsi tiketnya.
- **TiketReguler.java (Concrete Subclass):** Merupakan turunan langsung dari TiketBioskop yang merepresentasikan studio reguler dengan atribut tambahan nomorStudio. Memanggil super(judulFilm, hargaDasar, jumlahKursi) pada konstruktor dan mengimplementasikan logika perhitungan tarif standar (harga dasar dikalikan jumlah kursi).
- **TiketVIP.java (Concrete Subclass):** Merupakan turunan langsung dari TiketBioskop yang merepresentasikan studio mewah dengan atribut biayaFasilitasVIP. Biaya fasilitas recliner/lounge dihitung untuk setiap kursi, sehingga rumus total bayar diimplementasikan menjadi (hargaDasar + biayaFasilitasVIP) * jumlahKursi.
- **PaketSnack.java (Concrete Class):** Merupakan kelas mandiri di luar rumpun tiket bioskop yang secara langsung merealisasikan antarmuka ItemTransaksi. Memiliki atribut terenkapsulasi namaPaket, jumlahPorsi, dan hargaPerPorsi, serta membuktikan bahwa interface mampu menyatukan entitas bisnis yang berbeda silsilah keluarga ke dalam satu mekanisme kasir.

- **Main.java (Driver Execution):** Menguji proteksi kelas abstrak (pembuktian new TiketBioskop(...) dinonaktifkan sebagai komentar), memanggil static method antarmuka untuk mencatat log pembukaan transaksi kasir Pelanggan #042, menampung tiket reguler, tiket VIP, dan paket snack ke dalam array polimorfik heterogen ItemTransaksi[], mengiterasi pemanggilan default method cetakRincianItem(), mengakumulasikan total belanjaan, dan menutup transaksi.

D. Kesimpulan

Berdasarkan materi teori, latihan guided (gd1, gd2, gd3), dan pengerjaan tugas unguided, dapat ditarik beberapa kesimpulan penting:

- **Fungsi Kelas Abstrak (Abstract Class):** Kelas abstrak digunakan sebagai cetak biru (blueprint) kelas induk yang tidak dapat diinstansiasi secara mandiri, bertujuan untuk mendefinisikan state dan perilaku dasar bersama (partial implementation) sekaligus memaksakan kontrak method abstrak yang wajib diwujudkan oleh kelas-kelas turunannya.
- **Peran Antarmuka (Interface):** Interface bertindak sebagai kontrak perilaku murni (pure contract) yang mengatasi batasan pewarisan tunggal (single inheritance) di Java melalui multiple realization, serta memungkinkan penggabungan entitas lintas hierarki (seperti tiket bioskop dan makanan/minuman) ke dalam satu sistem pemrosesan.
- **Modernisasi Interface Java 8+ (Default & Static Method):** Kehadiran default method memungkinkan pengembang menyuntikkan fungsionalitas bersama ke dalam interface tanpa merusak (breaking) kelas-kelas yang telah mengimplementasikannya, sementara static method menyediakan fungsi utilitas independen yang dapat dipanggil langsung dari antarmuka.
- **Polimorfisme Heterogen (Dynamic Method Dispatch):** Kombinasi interface dan abstract class membuka kemampuan polimorfisme tingkat tinggi, di mana berbagai objek dari hierarki keluarga yang berlainan dapat disimpan dalam satu koleksi statis bertipe antarmuka umum (ItemTransaksi[]) dan dieksekusi perilakunya secara dinamis saat runtime.

Abstraksi melalui kelas abstrak dan antarmuka merupakan pilar utama dalam perancangan perangkat lunak berorientasi objek untuk menciptakan sistem yang modular, memiliki keterikatan longgar (loosely coupled), konsisten terhadap kontrak, serta mudah diperluas (extensible) di masa mendatang.